AP CSA Writing Methods: Rules, Patterns & Traps

Signatures, returns, parameters, six patterns, library calls (Math), and the traps

Method questions are graded on three things: the **signature** (did you copy it exactly?), the **return type** (does every path return the right thing?), and the **behavior** (does it do what the prompt says, on every input?). Everything below maps to one of those three.

1. Anatomy of a Method

```
public static int countVowels(String s) {
//  ^      ^     ^   ^          ^
//  |      |     |   |          +-- parameter list: type + name, comma separated
//  |      |     |   +------------- name: camelCase, starts with a verb
//  |      |     +----------------- return type (or void)
//  |      +----------------------- static: belongs to the class, no object needed
//  +---------------------------- access: public so the driver can call it
}
```

Part	Rule	Failure mode
Access	`public` for anything the driver calls	`private` method is invisible outside the class
Static	Only if it needs no object state	Calling `this` inside = CE
Return type	Must match the value returned	Missing or wrong type = CE
Name	Same name in the prompt, spelled identically	"cannot find symbol" at the call site
Parameters	Copy types **and order** verbatim	Same types in swapped order compiles but fails tests

2. Return Rules

- **Every path** of a non-`void` method must return. A method that returns only inside an `if` or a loop does not compile: `CE: missing return statement`.
- A `void` method returns nothing: `return;` is legal, `return value;` is `CE: incompatible types`.
- A `void` method's result cannot be used: `int x = printStuff();` is a CE.
- Widening is automatic (`int` -> `double`); narrowing is not (`return 2.5;` in an `int` method is a CE, you need an explicit cast and you lose the fraction).
- `return` exits the method immediately. Any statement after it in the same block is unreachable -> `CE: unreachable statement`.
- A `return` inside a loop ends the whole method. That is exactly what "find the first match" needs.
- Multiple `return` statements are legal and usually cleaner than a boolean flag.

```
// WRONG: compiles-looking but does not compile (missing return)
public static int indexOf(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) return i;
    }
    // nothing here -> CE: missing return statement
}

// CORRECT: sentinel return after the loop
public static int indexOf(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) return i;
    }
    return -1;
}
```

3. Parameter Passing: The Highest-Value Trap

Java is **pass-by-value, always**. When the parameter is an object, the value copied is the *reference*, not the object.

- **Reassigning the parameter** inside the method: the caller sees nothing.
- **Mutating the object** the parameter points to: the caller sees everything.

```
public static void editContents(int[] a) { a[0] = 99; }   // caller's array changes
public static void pointElsewhere(int[] a) { a = new int[3]; } // caller unaffected
public static void editPrimitive(int n) { n = 99; }        // caller unaffected
```

Argument passed	What the method can change	What it cannot change
`int`, `double`, `boolean`, `char`	nothing outside its own frame	the caller's variable
`String`	nothing (immutable)	the caller's `String`; must `return` a new one
array	elements (`a[i] = ...`)	which array the caller's variable names
`ArrayList`	contents (`add`, `remove`, `set`)	the caller's reference
custom object	its fields through public methods	the caller's reference

> Consequence for "write a method that..." questions: if the prompt says *returns a new String/array/list*, build and return it. If it says *modifies the given list*, mutate it in place and return `void`.

4. Locals, Fields, and Shadowing

- A parameter with the same name as a field **shadows** it; `name = name;` assigns the parameter to itself and does nothing. Use `this.name = name;`.
- Locals must be assigned before use: `int total; total += 5;` is `CE: variable total might not have been initialized`.
- Accumulators need a starting value that suits the job: `sum = 0`, `count = 0`, `max = arr[0]` (not `0`, which breaks all-negative data), `found = false`, `result = ""`.
- A local declared inside a loop body is recreated each iteration - that is why "declare before the loop" is required when you accumulate.

```
public static double average(int[] scores) {
    if (scores.length == 0) return 0.0;   // guard: never divide by zero
    int sum = 0;                          // declared before the loop
    for (int s : scores) sum += s;
    return (double) sum / scores.length;  // cast BEFORE dividing
}
```

5. Overloading vs. Overriding

	Overloading	Overriding
Where	Same class, same name	Subclass replaces a superclass method
Must differ in	parameter list (count, types, order)	nothing - signature must be identical
Return type	May differ, but that alone does not overload	Must be the same, or covariant
Chosen	Compile time, by the static types of the arguments	Runtime, by the actual object's class

- `public int f(int x)` plus `public double f(int x)` is a **duplicate method** -> CE. Return type is not part of the signature.
- `f(5)` with both `f(int)` and `f(double)` picks the exact match: the `int` version.
- Changing a parameter type in a subclass creates an overload that nobody calls - it looks like a working override and silently does nothing. This is the nastiest inheritance bug on the exam.

6. The Six Method Patterns You Actually Write

```
// 1. Count matches
public static int countAbove(int[] arr, int limit) {
    int count = 0;
    for (int v : arr) if (v > limit) count++;
    return count;
}

// 2. Sum and average (cast before dividing)
public static double mean(double[] nums) {
    if (nums.length == 0) return 0.0;
    double sum = 0.0;
    for (double n : nums) sum += n;
    return sum / nums.length;
}

// 3. Max / min: start from the first element, not from 0
public static int maxValue(int[] arr) {
    int best = arr[0];
    for (int i = 1; i < arr.length; i++) if (arr[i] > best) best = arr[i];
    return best;
}

// 4. Find / return -1
public static int findName(String[] names, String target) {
    for (int i = 0; i < names.length; i++) {
        if (names[i].equals(target)) return i;   // .equals, never ==
    }
    return -1;
}

// 5. Build and return a new collection
public static ArrayList<String> longWords(ArrayList<String> words) {
    ArrayList<String> out = new ArrayList<String>();   // construct it!
    for (String w : words) if (w.length() > 5) out.add(w);
    return out;
}

// 6. Modify in place (void)
public static void doubleAll(int[] arr) {
    for (int i = 0; i < arr.length; i++) arr[i] *= 2;
}
```

7. Recursive Method Notes

- Write the **base case first**, then assume the recursive call already works.
- Verify the argument actually shrinks toward the base case (`n - 1`, `s.substring(1)`, `arr.length - 1`). A call that does not shrink is infinite recursion -> `StackOverflowError`.
- Decide whether you **combine** the result (`return n + rec(n - 1)`) or **return it directly** (`return rec(n - 1)`); mixing them up turns a sum into a single step.
- String recursion must guard the empty string before touching index 0: `if (s.isEmpty()) ...` or `if (s.length() == 0) ...`.
- Trace with a tiny input (0, 1, 2) on paper; the stack depth equals the number of calls.

```
public static String reverse(String s) {
    if (s.length() <= 1) return s;              // base case first
    return reverse(s.substring(1)) + s.charAt(0); // combine on the way back up
}
```

8. Calling Library Methods: Read the Signature First

You never write these methods, so the only thing graded is whether your **call** matches the signature: right number of arguments, right types, and a return value you actually cast or store correctly.

The `Math` class: static, so no `new`

Call	Returns	What people get wrong
`Math.abs(int)` / `Math.abs(double)`	`int` / `double`	Returns the **same type** you passed; `Math.abs(-7)` is `7`, `Math.abs(-2.5)` is `2.5`
`Math.pow(double, double)`	`double`	`Math.pow(2, 3)` is `8.0`, not `8` - assigning it to an `int` is a CE
`Math.sqrt(double)`	`double`	`Math.sqrt(9)` is `3.0`; perfect squares do not come back as `int`
`Math.random()`	`double` in `[0.0, 1.0)`	`1.0` is never returned; the cast has to wrap the whole product
`Math.max(a, b)` / `Math.min(a, b)` *(extra)*	same type as the arguments	Mixing `int` and `double` promotes the result to `double`: `Math.max(3, 4.5)` is `4.5`, so it will not fit in an `int`
`Math.round(double)` *(extra)*	`long`	Returns `long`, not `int` - needs a cast before it can go into an `int`
`Math.PI` *(extra)*	`double` constant field	A field, no parentheses: `Math.PI`, never `Math.PI()`

The quick reference prints `abs`, `pow`, `random`, and `sqrt`. Rows marked *(extra)* are real Java but not on the printed sheet - only use them if the prompt does.

```
int a = Math.abs(-7);            // 7      (int in, int out)
double b = Math.sqrt(16);        // 4.0    (always a double)
int c = (int) Math.pow(2, 10);   // 1024   cast required: double -> int
int d = Math.pow(2, 10);         // CE: incompatible types, double cannot become int
```

The three cast traps

1. **Return type is `double`, so the assignment is a CE.** `int n = Math.pow(3, 2);` does not compile. Java will not quietly drop the fraction: insert `(int)`.
2. **The cast binds to what is directly after it.** `(int) Math.random() * 10` casts the fraction (`0.0 -> 0`) and then multiplies: the answer is **always 0**. Wrap the product: `(int) (Math.random() * 10)` gives 0-9.
3. **Casting truncates, it does not round.** `(int) 9.99` is `9`. If the prompt wants rounding, use `Math.round(...)` and cast its `long` result.

Random ranges worth memorising

Want	Write
A die: 1 to 6	`(int) (Math.random() * 6) + 1`
An index: 0 to n-1	`(int) (Math.random() * n)`
Any int from `min` to `max` inclusive	`(int) (Math.random() * (max - min + 1)) + min`

Because `Math.random()` never returns exactly `1.0`, the `+ 1` in the die formula is what makes 6 reachable. Drop it and you get 1-5.

Integer division happens inside the arguments

Arguments are evaluated **before** the call, so the usual integer-division rule still applies:

```
Math.pow(5 / 2, 2)   // 4.0,  not 6.25 — 5 / 2 is int division (2) first
Math.sqrt(9 / 2)     // 2.0,  not 2.121 — 9 / 2 is 4
Math.pow(5 / 2.0, 2) // 6.25 — one double operand fixes it
```

Other library calls with the same discipline

Call	Returns	Trap
`s.length()`	`int`	Method, with `()`; arrays use `arr.length` with none
`s.substring(a, b)`	`String`	`b` is **exclusive**; `b > s.length()` throws `StringIndexOutOfBoundsException`
`s.indexOf(t)`	`int`	Returns `-1` when absent - check before feeding it to `substring`
`s.charAt(i)`	`char`	`i` must be `0 .. length()-1`; `char` comparisons use `==`, `String` comparisons do not
`s.equals(t)` / `s.compareTo(t)`	`boolean` / `int`	`compareTo` gives a **sign** (negative, zero, positive), not a distance

Call	Returns	Trap
`list.size()`	`int`	Method, with `()`; no `.length` on an `ArrayList`
`list.get(i)` / `list.set(i, v)`	element type	`get` returns, `set` returns the **old** value
`list.remove(i)` vs `list.remove(obj)`	element / `boolean`	With an `Integer` list, `remove(2)` removes the *index*; remove by object needs a cast
`Integer.parseInt(s)` *(extra)*	`int`	Throws `NumberFormatException` on non-numeric text
`s.trim().toLowerCase()`	`String`	Chaining runs left to right; each call returns a **new** String (Strings are immutable)

Four habits for any library call

- Read the parameter list first (count and types), then the return type. Most wrong answers are a type mismatch, not a logic error.
- Static library methods are called on the **class**: `Math.pow(...)`, never `myMath.pow(...)` - you cannot `new Math()`.
- Cast the whole expression, not the first token.
- Check `-1`, `0`, or `null` before using a returned value as an index or a divisor.

9. Method Traps Checklist

Trap	Symptom	Fix
Return only inside a branch	`CE: missing return statement`	Add a final `return` after the loop
`void` + `return value;`	`CE: incompatible types`	Drop the value or change the return type
Reassigning an object parameter	Caller unchanged, tests fail	Mutate the object, or return the new value
`==` on Strings/objects	Wrong answer on equal contents	`.equals()`
Integer division	`2` instead of `2.5`	`(double) a / b`
Uninitialized accumulator	`CE: might not have been initialized`	Initialize before the loop
`max` starts at `0`	Fails on all-negative arrays	Start at `arr[0]`
Calling a method without `()`	`CE: cannot find symbol` (it is a variable)	Add arguments
Instance method from static `main`	`CE: non-static method cannot be referenced`	`new Obj().method()` or make it static
Same name, different return type	`CE: duplicate method`	Change the parameter list instead
Method call on `null`	`NullPointerException` at runtime	Guard with `!= null` before the call
`int n = Math.pow(2, 3);`	`CE: incompatible types` (double into int)	`(int) Math.pow(2, 3)`
`(int) Math.random() * 10`	Always `0`: the cast hits the fraction first	`(int) (Math.random() * 10)`
`int m = Math.max(3, 4.5);`	`CE`: the mixed argument promotes it to `double`	Keep both arguments `int`, or cast the result
`Math.pow(5 / 2, 2)`	`4.0` instead of `6.25`	One double operand: `5 / 2.0`

10. Before You Submit

- [ ] Signature matches the prompt character for character (name, parameter types, order, return type).
- [ ] Non-`void`: a `return` on every path, including empty-input and no-match paths.
- [ ] `void`: no `return value;`, no `System.out.print(voidCall())`.
- [ ] Every accumulator initialized, and initialized to the right seed value.
- [ ] Objects mutated in place only when the prompt asks for in-place changes; otherwise return a new one.
- [ ] Loop bounds use the right length: `arr.length` (array) vs `list.size()` (ArrayList).
- [ ] `.equals()` for contents, == only for primitives and `null` checks.
- [ ] Cast where a type changes: before dividing for a decimal result, and around `Math.pow`/`sqrt`/`random` before storing in an `int`.
- [ ] Random range double-checked: `(int) (Math.random() * n) + min`, with the cast around the whole product.
- [ ] Traced once by hand with a 0-element, 1-element, and normal input.